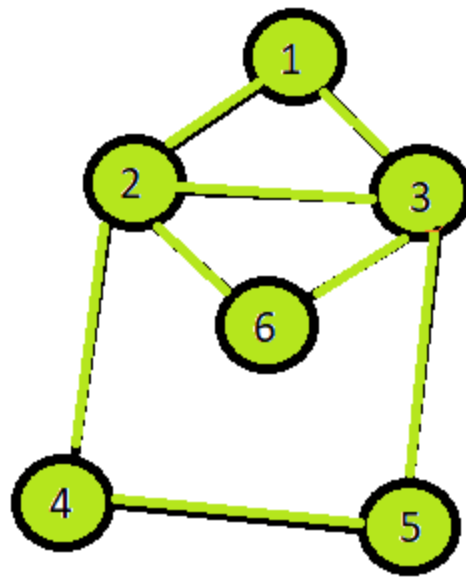


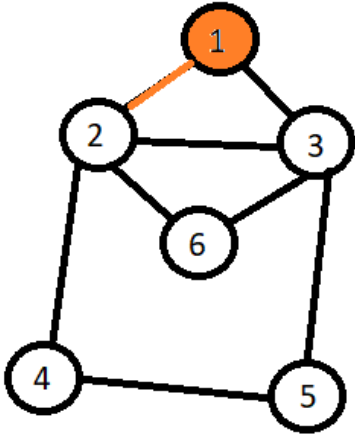
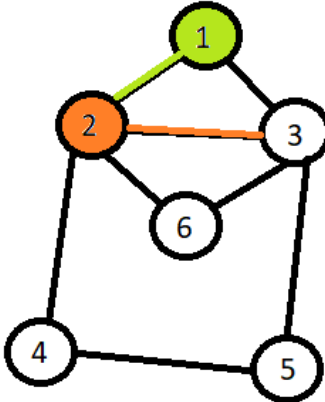
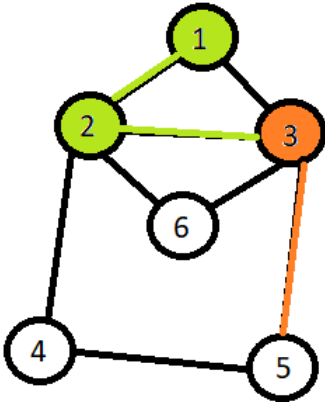


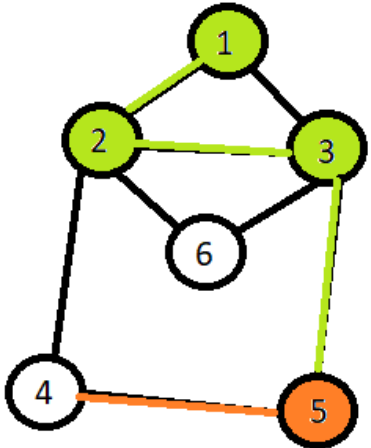
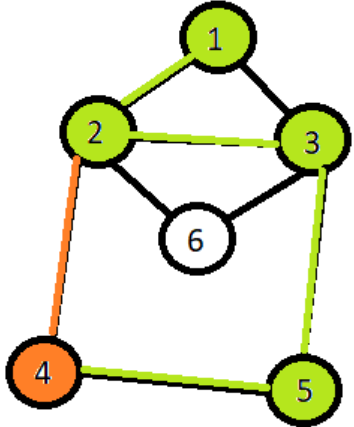
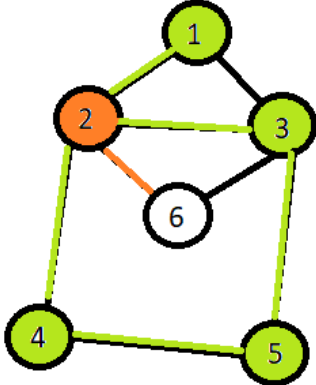
Source for test case: <https://cses.fi/problemset/task/1691>

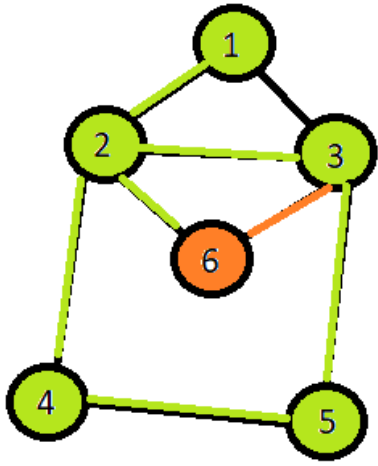
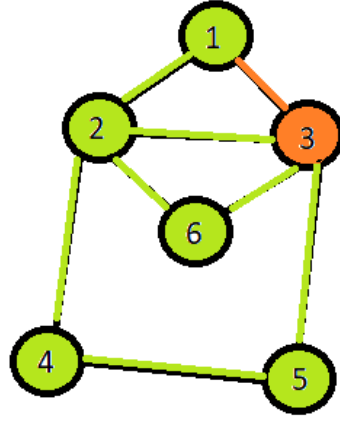
a. Algorithm Implementation

1. Fleury's Algorithm

Step	Edges	Graph
0	-	

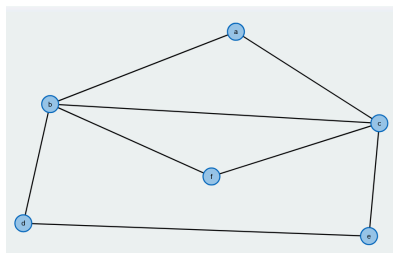
1	1-2 ✓	
	1-3	
2	2-3 ✓	
	2-4	
	2-6	
3	3-5 ✓	
	3-6	

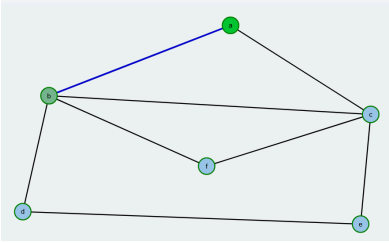
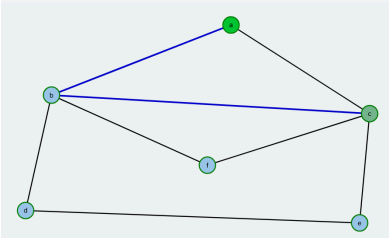
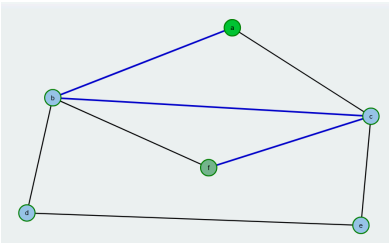
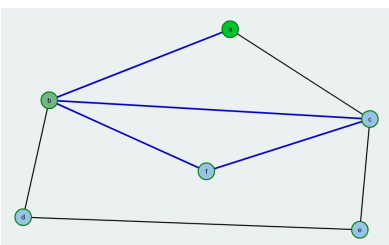
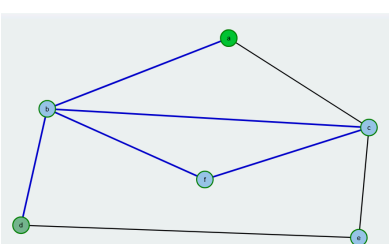
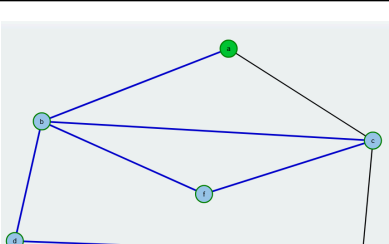
4	5-4 ✓	
5	4-2 ✓	
6	2-6 ✓	

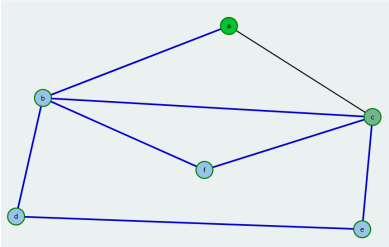
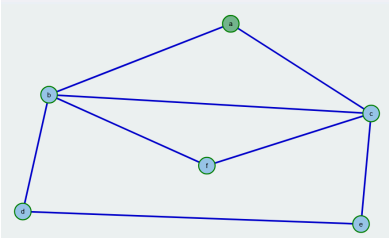
7	6-3 ✓	
8	3-1 ✓	

2. Hierholzer's Algorithm*

The nodes have been replaced with alphabets, so it starts from node 'a'

Step	Traversed Edges	Graph
0	-	

1	a-b (1-2)	
2	b-c (2-3)	
3	c-f (3-6)	
4	f-b (6-2)	
5	b-d (2-4)	
6	d-e (4-5)	

7	e-c (5-3)	
8	c-a (3-1)	

3. Tarjan's Algorithm*

*The nodes have been shifted to start from 0

0 -> 1

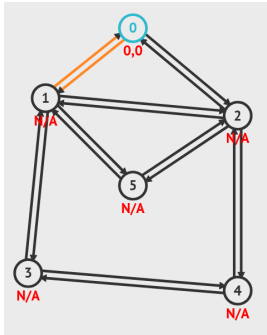
1 -> 2

2 -> 3

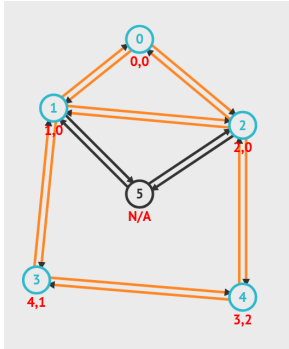
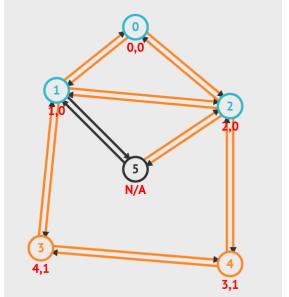
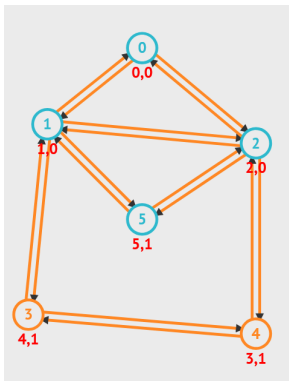
3 -> 4

4 -> 5

5 -> 6

Steps	Visited Node	Traversed Edges	(discovery_order, nearest_loop_node)	Graph
1	0	0-1 (1-2)	(0,0)	

2	1	1-2 (2-3)	(1,0)	
3	2	2-0 (3-1)	(2,0)	
4	2	2-4 (3-5)	(2,0)	
5	4	4-3 (5-4)	(3,2)	

6	3	3-1 (4-2)	(4,1)	
7	Return to 4	2-5 (3-6)	(3,1)	
8	5	5-1 (6-2)	(5,1)	

b. Analysis

Based on time complexity, Hierholzer's algorithm is the most efficient choice for the Eulerian cycle problem. It has a time complexity of $O(E)$, so it performs in linear time depending on the number of edges. Every edge is visited and processed once. Fleury's algorithm has a time complexity of $O(E^2)$, at every step of the path, it checks if an available edge is a bridge (an edge that disconnects the remaining unvisited graph if removed), which results in slower traversal time for Eulerian cycle problems. Tarjan's algorithm has a time complexity of $O(V+E)$, like Hierholzer's, this algorithm also performs in linear time, but it is a bridge-finding algorithm. It would improve the efficiency of Fleury's algorithm, but it is not optimal when used alone for the test case used here.